SDP

From Prompts to Skills

⏰ Build it once. Invoke it anywhere. Graduate it to a skill.

Part 1: save your best prompts as reusable team functions. Part 2: embed them into the workflow as agent skills.

M1 Instructions › M2 Plan Mode › **M3 Prompts** › **M4 Skills** › M5 MCP › M6 Agents

⏰ : Monday, 12:30 PM

E **Elena** wants to capture and maintain high-value prompts for reuse across the team.

> " Every review I run from memory is a potential gap. When the prompt lives in my head, the checklist stops the moment I'm not in the room. If we codify these, the whole team runs the same quality bar."

Module 1

Document the standards

Architecture docs and repository instructions became the source of truth.

Module 2

Improve the workflow

Plan mode made complex work reviewable before code started landing.

Module 3

Save the best prompts

Now the goal is turning repeated, high-value prompting into something the whole team can invoke.

⚡ **Custom prompts turn personal prompt craft into invokable team workflow.**

💭 The Team, Before Module 3+4

⚠ WRONG ANSWERS ONLY

Jessica

The Eager Junior

"A skill is a really long prompt. Like, a detailed one you save so you don't have to retype it. That's it."

Underselling it by about ninety percent.

David

The Seasoned Architect

"Prompt files are macros. We solved this in 1980. I'm going to need a more compelling pitch."

Waiting for the pitch. About to get one.

Sarah

The Skeptical Senior

"Skills are pre-loaded context. You write it once and Copilot remembers it forever. Low maintenance."

The 'forever' is doing a lot of heavy lifting.

Elena

The Quality Champion

"A prompt file is always active. It runs on every request. You can't turn it off without deleting it."

Already planning how to audit the always-on behavior.

Marcus

The DevOps Developer

"Skills are aliases. You type the name, it expands to the full prompt. Like a bash alias but for chat."

Close enough to feel right. Wrong enough to matter.

Rafael

The Product Visionary

"I turn our PRD template into a skill and it writes the whole spec from a one-line prompt. That's the use case."

He's going to be partially right. That's almost worse.

What You'll Build

Five exercises. Two modules. One complete workflow layer.

Prompts capture repeatable tasks — skills embed them into the workflow

PART 1 — PROMPT FILES

E1 — Refresh the source of truth

`refresh-docs.prompt.md`

Use `/create-prompt` to build a reusable prompt that keeps `ARCHITECTURE.md` and team instructions aligned after code changes.

E2 — Save one strong prompt

`good-idea-fairy / risk-prioritizer / commit-helper`

Use `/create-prompt` to pick one: generate feature ideas, prioritize risks, or automate branch + PR creation.

E3 — Encode domain knowledge

`check-data-accuracy.prompt.md`

Use `/create-prompt` to build a generic accuracy checker that applies to any entity type and upgrades to live DB queries via MCP in Module 5.

PART 2 — AGENT SKILLS

E4 — Graduate a prompt to a skill

`/create-skill → check-data-accuracy/`

Use `/create-skill` to promote the E3 prompt to a skill Copilot reaches for automatically — no manual invocation needed.

E5 — Card & page skill

`/create-skill → create-card-and-page/`

Use `/create-skill` to graduate the M2 plan into a parameterized skill for any FanHub entity type.

Prompts capture your intent. Skills make that intent *available to Copilot* — so it can reach for the right behavior automatically, not just when you remember to ask.

What Prompt Files Actually Are

Think of them like functions

- Invoked on demand
- Named and discoverable
- Parameterized with variables
- Designed for specific repeatable tasks

Each file has two layers

- YAML frontmatter for name, agent, model, tools
- A body that explains the task and expected output
- Optional links to docs and standards
- Optional built-in or input variables

Workspace prompts

- Live in `.github/prompts/`
- Are version-controlled and team-visible
- Work best for project-specific workflows

User prompts

- Live in your VS Code profile
- Follow you across projects
- Work best for personal, reusable habits

> *"Named, discoverable, parameterized, invoked on demand... that's a function. Why did nobody just say it was a function? I know how to write functions."*
>
> — Jessica

🔗 Why Prompt Files Matter Here

⏳

Less retyping

The team stops rebuilding the same careful prompt every time a familiar task appears. Write it once, invoke it forever.

📐

Less standards drift

Prompts can reference current architecture and instruction files instead of embedding stale assumptions copied from memory.

🤝

More reusable expertise

A strong review or debugging prompt becomes something the entire team can benefit from — not just the person who wrote it.

Prompt files make standards easier to invoke than to forget.

What a Strong Prompt File Includes

Frontmatter

- A clear name and description
- The right agent for the task
- Tool access that matches the prompt's job
- Optional hints that improve invocation

Body

- The exact task to perform
- Links to standards or examples when relevant
- The outputs you expect back
- Enough structure to keep results consistent

Variables

- `${file}` for current file context
- `${selection}` for focused analysis
- `${workspaceFolder}` for project-aware prompts
- `${input: ... }` for parameterized use

Good design rule

The best prompt file is specific enough to save time, but general enough that the team actually uses it more than once.

> *"Specific enough to save time, general enough to use more than once. I've been saying that about functions since 2010. Glad the prompt file agrees."*
>
> — Sarah

📊 What to Validate in the Workshop

Invocation speed

Compare writing a repeated prompt manually with invoking the saved version.

Standards alignment

Check whether linked instructions and architecture docs remove missed checks and stale wording.

Context capture

Use variables to reduce manual copying and improve prompt consistency during debugging or review.

Prompt files pay off twice: once in the time you save, and again every time a teammate runs the same quality bar without asking you how.

🔨 Exercises — Prompts

3 exercises · ~28 minutes · shared leadership

E1 Rafael ⭐

Refresh the docs

Build a prompt that reviews the workspace and updates only what changed after any meaningful code push. Run it after every sprint — docs stay current without anyone tracking it manually.

"Docs drift is a sprint velocity OKR. I'm logging it."

E2 Jessica ⭐

Save one strong prompt

Pick one type, write it, save it — validate it solves something real.

🧚	**Good Idea Fairy**	— creative feature ideas
⚠️	**Risk Prioritizer**	— ranked fixes by risk
🔀	**Commit Helper**	— branch, commit, PR

E3 Marcus ⭐

Encode domain knowledge

Build a check-data-accuracy prompt that validates any data entity — lore, characters, episodes — against its canonical source before anything hits review. MCP upgrades this to live DB queries in Module 5.

"I had a tab open. Wait. I spent an hour checking that PR manually?"

Each exercise rotates the lead persona — everyone contributes, [illegible]

EXERCISE 1

E1 Creating a Docs Refresh Prompt

Rafael ⭐ ⏰ 6 min

Rafael's already thinking about the next sprint. He's noticed that every time the team plans a feature, Copilot's suggestions drift slightly from what the codebase actually is — because nobody updated the docs after Module 2. *That drift compounds.* **He wants one prompt he can hand to the team: run it after any meaningful change, docs stay current, problem never comes back.**

📋 STEPS

1. Run `/create-prompt` in Copilot Chat to scaffold `refresh-docs.prompt.md` — describe what you want it to do and let Copilot generate the file
2. Use `/plan` and ask for a review the current workspace and update `ARCHITECTURE.md` and `copilot-instructions.md` only when the codebase has changed
3. Save it, and invoke it after each change to keep the docs aligned with current reality instead of stale memory

✅ SUCCESS CRITERIA

- ☐ The prompt is reusable after any meaningful code change
- ☐ It updates the two source-of-truth docs without turning into a generic repo rewrite
- ☐ The team sees evergreen docs as [illegible]

💭 **Rafael:** *"The docs drift is a compounding risk. One prompt maintained. I'm already thinking about what we build next."*

Example: The refresh-docs Prompt

Type `/create-prompt` in Copilot Chat, then describe it:

```
User >

/create-prompt Create a prompt called refresh-
docs that reviews the workspace and proposes
targeted updates to ARCHITECTURE.md and
.github/copilot-instructions.md — only
sections that are inaccurate or missing. Use
the plan agent so changes are always reviewed
before anything is written.
```

How it works

1. Run `/create-prompt` in Copilot Chat — describe what you want it to do and tell it to use `agent: plan`
2. Run `/refresh-docs` in Copilot Chat
3. The Plan agent runs automatically — producing a reviewable plan before anything changes
4. Review the plan, approve, execute

Why this matters

The `agent: plan` in the prompt is what makes it safe — it never blindly rewrites docs. You always see what it wants to change before a single file is touched.

Marcus: *"Wait, I just describe what I want and it generates the whole prompt file? Including the plan agent bit? I could get used to this. Anyway — I was reading this Hacker News thread about—"*

E2

EXERCISE 2

Save One Strong Prompt

Jessica ⭐ ⏰ 10 min

Jessica has been quietly watching Rafael and Marcus work. She's convinced everyone else is naturally good at this. Then she sees the option list. *Any of these could be hers.* **Pick one prompt type, write it, save it to `.github/prompts/`, and validate it actually solves something real.**

Good Idea Fairy
Generate bold, creative feature ideas for the FanHub site — the kind no one's thought to ask for yet

⚠️ **Risk Prioritizer**
Review code, data, and architecture to produce a prioritized list of fixes ranked by actual risk

🔀 **Commit Helper**
Create a branch if needed, commit the changeset, and open a well-formed PR — all in one go

STEPS

1. Pick one of the three prompt types above
2. Run `/create-prompt` in Copilot Chat to scaffold the `.prompt.md` — give it a clear description and focused instructions
3. Save to `.github/prompts/` and invoke it — does it do what you needed?

✅ SUCCESS CRITERIA

- ☐ Prompt is saved and invokable by name
- ☐ Output is useful — not just [illegible]
- ☐ You'd actually use this [illegible]

💭 ***Jessica:*** [illegible]

Example: Good Idea Fairy

Type `/create-prompt` in Copilot Chat, then describe it:

```
User >

/create-prompt Create a prompt called good-idea-fairy that reads ARCHITECTURE.md and generates 5 bold, specific feature ideas for FanHub — each with a punchy name, one-sentence description, why fans would love it, and any technical considerations. Use the plan agent.
```

How it works

1. Save to `.github/prompts/`
2. Run `/good-idea-fairy` in Copilot Chat
3. Copilot reads your architecture first — ideas are creative *and* grounded

Why this matters

Grounding ideation in `ARCHITECTURE.md` means suggestions fit your actual stack — not generic advice for a generic app.

[illegible]: *"It knew we didn't have a notification system yet. I didn't tell it that. It just... read the file."*

⚠ Example: Risk Prioritizer

Type `/create-prompt` in Copilot Chat, then describe it:

```
User >

/create-prompt Create a prompt called risk-
prioritizer that reviews the codebase and
ARCHITECTURE.md for top risks across security,
data integrity, scalability, and unhandled
failures — each with a severity rating and
recommended fix, sorted by severity. Use the
plan agent.
```

How it works

1. Save to `.github/prompts/`
2. Run `/risk-prioritizer` in Copilot Chat
3. Returns a ranked, actionable list — not a generic checklist

Why this matters

"Review for risks" without structure returns noise. Categories and severity rubric force Copilot to reason, not just list.

💬 **Jessica** *"It found the unhandled auth timeout I'd been meaning to check for three sprints. Just… there it was. Top of the list."*

🔀 Example: Commit Helper

Type `/create-prompt` in Copilot Chat, then describe it:

```
User >

/create-prompt Create a prompt called commit-
helper that looks at current git changes,
creates a feature branch if needed, stages
changes, writes a conventional commit message,
pushes, and opens a PR with a clear title and
description — asking for confirmation before
the PR is created. Use the agent mode.
```

How it works

1. Save to `.github/prompts/`
2. Run `/commit-helper` in agent mode
3. Copilot handles branch → commit → PR and asks for confirmation before submitting

Why this matters

The confirmation step keeps you in control. You get the automation without losing the review — Copilot prepares, you approve.

💬 **Jessica:** *"It wrote a better PR description than I would have. And it remembered to create the branch first — I always forget the branch."*

EXERCISE 3

E3

Data Accuracy Check

Marcus ⭐ Sarah 🤝 ⏰ 12 min

Marcus shipped the `/lore` page last sprint. Now he's batch-generating entries with Copilot — ten show facts in fifteen minutes. The PR looks fine. Sarah reviews it and finds errors. She flags them. She is not doing this every sprint. **Marcus needs a prompt that validates any data entity — lore entries, character bios, episode records — against the canonical source before it goes into the database.** In Module 5, MCP gives that source live DB access. For now, the reference is a markdown file.

📋 STEPS

1. Run `/create-prompt` in Copilot Chat to scaffold `check-data-accuracy.prompt.md` — tell it to use `${selection}` for the content and `${input:entityType}` for the domain, or let Copilot infer it
2. Reference your canonical source file (e.g. `docs/[show]-universe.md`) — no hardcoded entity names in the prompt body
3. Test: select a known-bad entry → prompt must flag the error with the correct value
4. Test: select a correct entry → prompt must pass with no

✅ SUCCESS CRITERIA

- ☐ Prompt saved under `.github/prompts`
- ☐ `${selection}` used — no hardcoded entry names
- ☐ Works on at least two different entity types (e.g. lore + character)
- ☐ Correct entry passes with no false positives

> 💭 **Marcus:** *"Wait — if the canonical rules are in a reference file, I just... point the prompt at each entry? And in Module 5, that reference becomes a live DB query?"*

📋 Example: check-data-accuracy.prompt.md

User >

`/create-prompt` Create a prompt called `check-data-accuracy` that verifies a selected text entry against the canonical reference file for that domain. It identifies the entity type from the selection, locate the relevant reference in the project, check each factual claim, and flag inaccuracies with specific corrections. Use the `ask` agent and respond with Status, Issues, and Verified claims.

Copilot generates `.github/prompts/check-data-accuracy.prompt.md` with `agent: ask`, `${selection}` as the input variable, and structured output format.

🔗 No hardcoded domain

Copilot infers entity type from the selection — works for lore, characters, episodes, ratings, or any new entity

✂️ ${selection}

Select any entry in the editor, invoke — no copy-paste needed

🗄️ MCP-ready

In Module 6, [illegible]

💡 Then invoke it

Select any entry in the editor and run `/check-data-accuracy`

💬 Example: Step 3 — Test an Inaccurate Entry

Demo domain: Breaking Bad lore. Select this entry in your editor, then run `/check-data-accuracy`:

```
/check-data-accuracy

Title: "Walter & Jesse — Origin"

Walter White recruited Jesse Pinkman as his partner after reconnecting
with him on the street. Jesse had been Walter's former student at the
University of New Mexico, where Walter taught a chemistry course.
```

> 💬 **David:** *"I'm going to stop you right there. Walter White has a Master's degree. From the show. That he teaches at a high school is, frankly, the entire dramatic irony of the series. This is not a subtle error."*

Example: Step 3 — Output (Error Flagged)

```
Status: Inaccurate

Issues:

Walter White taught at J.P. Wynne High School — not the University of New Mexico. He was a high school chemistry teacher, not a university lecturer.

Jesse Pinkman is described as Walt's former student, which is correct, but the institution is wrong. The connection is from high school, not a university.

Verified claims:

✓ Walter White and Jesse Pinkman are partners
✓ Jesse was Walter's former student
✓ Jesse was a small-time meth cook prior to their partnership
```

💭 ***Sarah:*** *"Yes, Marcus. Walter White was a high school teacher. This is literally the premise of the show. The prompt caught it. You're welcome."*

Example: Step 4 — Test a Correct Entry

Same demo domain. Select this entry and run `/check-data-accuracy` again — validate no false positives:

```
/check-data-accuracy

Title: "The Blue Sky Formula"

Walter White's methamphetamine, known on the street as "Blue Sky,"
achieved a purity level exceeding 99% — far beyond any competitor. Its
distinctive blue tint came from the synthesis method Walter developed at
the Superlab.
```

> ***Jessica:*** *"Wait, so if it passes that means it's... good? Like, Copilot checked it and it was right? That's so cool. I feel like we just gave the app a immune system."*

Example: Step 4 — Output (Partially Accurate / Unverifiable)

```
Status: Partially Accurate / Partially Unverifiable

Issues:

The exact purity figure in the reference is 99.1%, not just "exceeding 99%" — minor but worth being precise in a
lore entry.

The claim that the blue tint came from "the synthesis method Walter developed" is not verifiable from breaking-
bad-universe.md — the reference does not explain the chemical reason for the color.

Verified claims:

✓ The product is known as "Blue Sky" (implied by the blue color reference)
✓ Purity exceeds 99% — confirmed at 99.1% in the reference
✓ The product has a distinctive blue color — confirmed
✓ It was cooked in the Superlab — confirmed
✓ It became known worldwide — confirmed explicitly in the reference
```

Why this matters

A prompt that only flags violations is fragile — you need to confirm it doesn't false-positive on accurate content before trusting it

Now try it on your show

[illegible] `${selection}` [illegible]

💬 The Team, After the Prompt Exercises

Jessica

The Eager Junior

"Wait — I can just pick the one I need? The risk-prioritizer or the commit-helper? Even me? I don't have to write the whole thing from scratch every time?"

Realizing she can use the good version, not just her version.

David

The Seasoned Architect

"To be precise: this isn't automation. It's codification. We're encoding the decision, not removing it. That distinction matters."

Correcting the framing before conceding the point.

Sarah

The Skeptical Senior

"I checked. The check-data-accuracy caught the same error three times in a row. Three for three. Consistent beats clever. I'll take it."

Running the numbers before endorsing anything.

Elena

The Quality Champion

"Okay but I need edge cases. What if the lore entry is partially correct? Does it flag the wrong part? Show me the failure mode first."

Not satisfied until she's seen what breaks.

Marcus

The DevOps Developer

"Sorry — I was half-reading a thread about prompt files. Wait. We can check these in? They travel with the repo? That's actually... hold on."

Snapping into focus mid-sentence.

Rafael

The Product Visionary

"The data-check is already an OKR input. If canon accuracy goes up, lore engagement goes up. I'm mapping this to Q3. Can we move on?"

Already three steps ahead, agenda in hand.

Mindful Moment

✓ Custom Prompts →

▶ PART 2 — STARTING NOW
Agent Skills

Prompts Work. But They Don't Run Themselves.

Elena spent the morning running `/check-data-accuracy` before every PR. It worked perfectly — every time she remembered to invoke it.

A PROMPT IS INVOKED

- ✅ Accurate when you remember to run it
- ✅ Fast to create and iterate
- ✅ Great for on-demand tasks
- ⚠️ Depends on the human in the loop

A SKILL IS EMBEDDED ← WE'RE BUILDING THIS

- ✅ Available whenever Copilot works with lore content
- ✅ Cites sources automatically
- ✅ Runs as part of the workflow
- ✨ The domain knowledge is in the system

> "There's a distinction worth making. A prompt is a **what** — you define the output shape and invoke it when you need it. A skill is a **when and a how** — Copilot knows to use it, and knows which resources to consult. Elena built the what. **Now we build the when.**"
>
> D David, who would like the record to show he made this distinction before it became obvious

What Agent Skills Actually Are

A skill isn't a fancy prompt. It's a packaged capability — instructions, resources, and context Copilot reaches for automatically.

A capability package

- `SKILL.md` explains the workflow and goals
- Resources can include scripts, examples, schemas, and templates
- The goal is a complete task capability, not just a reminder

Progressive loading

- **Discovery:** metadata is known up front
- **Instructions:** the body loads when relevant
- **Resources:** scripts and examples load only when needed

Context-efficient by design

Project & personal scopes

- Project skills live in `.github/skills/`
- Personal skills live in your home skill directory
- Shared workflow belongs with the repo — not in someone's head

The differentiator

Skills go beyond "say this" and into **"use these resources, run this script, follow this workflow."**

Prompts you invoke. Skills Copilot reaches for.

Skill Structure and Loading

Skills are folders, not files — a structured package Copilot loads progressively as the task demands it.

Typical structure

```
.github/skills/check-data-accuracy/
  ├─ SKILL.md
  ├─ scripts/
  ├─ examples/
  └─ templates/
```

SKILL.md is the entry point — everything else loads on demand

Loading model

1. Copilot sees the **skill name and description** up front
2. The **body loads** when the task matches
3. **Referenced resources** load only on demand

✓ Many skills can exist without bloating every session

Progressive loading is what makes many skills practical instead of expensive.

> *"Lazy loading. For prompts. That's just lazy loading. I don't know why that makes me happy, but it does."*
>
> — Marcus

Skills vs Instructions vs Prompts

Three tools, three jobs. Each one belongs in a different place in your workflow.

Instructions

Always-on context

- Best for behavior and standards
- Often always-on or path-scoped
- Tell Copilot **how to work**

`.github/copilot-instructions.md`

Prompts

On-demand commands

- Best for repeated invocations
- Save and reuse strong request wording
- Turn common tasks into **/commands**

`.github/prompts/*.prompt.md`

Skills

Packaged capabilities

- Best for specialized capabilities
- Combine instructions with resources and scripts
- Teach **complete workflows** Copilot can apply

`.github/skills/*/SKILL.md`

Instructions guide behavior. Prompts package requests. Skills package capability.

📊 What to Validate in the Workshop

A skill that sounds right but doesn't improve the work isn't a skill — it's documentation. Test for real impact.

🎯 Capability fit

Check whether the skill actually **teaches the workflow**, not just describes it. Confirm Copilot can complete the task with the skill alone.

Does it change what Copilot can do?

📚 Resource usefulness

Validate that examples, templates, schemas, or scripts **materially improve** the output — not just add bulk to the context.

Does each resource earn its place?

⏱️ Time & rework saved

Compare specialized tasks **with and without** the skill in place. The delta is the proof.

Would you skip this skill next time?

A real skill changes the quality of work, not just the wording of the prompt.

🔨 Exercises — Skills

2 exercises · ~30 minutes · shared leadership

E4 Sarah ⭐

Graduate a prompt to a skill

Use `/create-skill` to promote check-data-accuracy from a prompt you invoke to a skill Copilot reaches for automatically.

"One command. It read the prompt, inferred the triggers, built the folder. I didn't write a single line of SKILL.md by hand."

E5 Marcus ⭐

Card creation as a repeatable skill

Take the M2 plan-mode output for creating lore cards and graduate it into a parameterized skill that generates a card + detail page for any FanHub entity type.

"Wait — I wrote the same plan twice. That's a pattern. Patterns become skills." [tab-switches, tab-switches back] *"Okay I'm back. Let's build it."*

Each exercise rotates the lead persona — everyone contributes, everyone follows.

E4

EXERCISE 4

Graduate a Prompt to a Skill

Sarah ⭐   ⏰ 10 min

The team has three prompt files from E1–E3. Sarah notices that `/check-data-accuracy` is being invoked before every PR — manually, every time. *A prompt you invoke and a skill Copilot reaches for automatically are not the same thing.* **Use `/create-skill` to graduate a prompt into a skill that triggers without being asked.**

STEPS

1. Run `/create-skill` in Copilot Chat — reference `check-data-accuracy.prompt.md` and ask it to promote the prompt to a skill that auto-triggers when verifying or fact-checking any data entry
2. Review the generated `.github/skills/check-data-accuracy/SKILL.md` — check the `description` trigger phrases cover verify, validate, fact-check for any entity type
3. Test: ask Copilot to `generate a data entry for a Quote` — does it cite the skill and reference source automatically?

☑ SUCCESS CRITERIA

- ☐ `.github/skills/check-data-accuracy/SKILL.md` exists with broad trigger phrases
- ☐ Copilot cites the skill when asked to write or check any data entry — without being explicitly invoked
- ☐ The team understands the difference: prompt = you invoke it, skill = Copilot reaches for it

💭 **Sarah:** *"One command. It read the prompt, inferred the triggers, built the folder. I didn't write a single line of SKILL.md by hand."*

💬 E4 Example — /create-skill in Action

💬 STEP 1 — THE PROMPT

```
/create-skill

Here’s my check-data-accuracy.prompt.md —
promote it to a skill named
check-data-accuracy that auto-triggers

when verifying or fact-checking any
data entry across lore, characters,
episodes, and ratings.

// VS Code built-in — no custom prompt needed
```

📋 STEP 2 — REVIEW THE OUTPUT

▶ RUNNING /CREATE-SKILL

→ Copilot reads `check-data-accuracy.prompt.md`, infers trigger phrases, scaffolds the skill folder

```
// .github/skills/check-data-accuracy/SKILL.md
---
name: check-data-accuracy
description: >
  Use this skill when asked to verify,
  validate, or fact-check any data entry —
  lore, characters, episodes, ratings.
  Triggers on: check data, verify entry,
  fact-check, accuracy check, is this correct.
---
 // body preserved from prompt file ...
```

💡 **Check the triggers.** [illegible] `description` [illegible]

EXERCISE 5

E5

Card Creation as a Repeatable Skill

Marcus ⭐ Jessica 🤝 ⏰ 15 min

In Module 2, plan mode produced the full roadmap for the Lore card and `/lore` page. Marcus saved it. Now he needs a Locations card. *The plan is almost identical.* **Generalize the M2 plan into a parameterized skill — one invocation creates the card + page plan for any FanHub entity.**

📋 STEPS

1. Run `/create-skill` in Copilot Chat — reference the M2 plan file and ask it to generalise it into a parameterised skill named `create-card-and-page`. Review the generated `SKILL.md` to confirm the skill name and folder
2. Ask Copilot to improve it with a `templates/` folder and a `scripts/` folder (see next two slides)
3. Test: `/create-card-and-page Locations`

✅ SUCCESS CRITERIA

- ☐ `create-card-and-page/` created with SKILL.md + templates/
- ☐ Skill accepts entity name and generates a scoped plan
- ☐ Plan identifies the same files as the [illegible] plan
- ☐ Templates folder [illegible]

💭 **Marcus:** *"Wait — I wrote the same plan twice. That's a pattern."* [tab-switches, tab-switches back] *"Okay I'm back. Patterns become skills. Let's build it."*

E5 Step 1 — Create the Skill

THE PROMPT

```
/create-skill

Use the updated Module 2 plan for the
Lore card + page, including the
add/edit form — @plan-
loreCardAndLorePageForm.prompt.md

Generalise this into a parameterised skill
named create-card-and-page that accepts
an entity name and produces an equivalent
scoped plan for any FanHub entity type.

Make sure the plan includes the form
alongside the card + page.

When the skill runs, it:

1. Ask the user what properties the entity
  has beyond title and description before
  generating anything.

2. Output a scoped plan and wait for
  approval before creating any files.
```

WHAT HAPPENS

! **Pre-flight:** make sure your M2 plan is updated to include the add/edit form before running this. The skill inherits whatever is in the plan.

→ Copilot reads the updated M2 plan, extracts the repeating structure, and scaffolds `.github/skills/create-card-and-page/SKILL.md`

→ The `description` field becomes the trigger: Copilot will reach for this skill whenever someone asks to add a new entity with a card grid page

→ **Ask questions:** when the skill runs it will prompt for extra properties before generating — no tools needed, just a chat turn

→ **Plan first:** the skill outputs a scoped plan and waits for approval — same muscle-memory as agent plan mode, built into every invocation

Marcus: [illegible]

📋 E5 Step 1 — Review the Generated Skill

.GITHUB/SKILLS/CREATE-CARD-AND-PAGE/SKILL.MD

```
---
name: create-card-and-page
description: >
  Use this skill when asked to add a new
  entity type with a card grid and detail
  page to FanHub. Triggers on: add entity,
  create card page, new card grid, scaffold
  entity, add ${entityName} to FanHub.
---

# create-card-and-page

Generate a scoped implementation plan for
adding ${entityName} as a card grid + detail
page.

## Plan
 1. Backend model + EF migration
 2. API controller + endpoints
 3. Card component + grid page
 4. Detail page + routing
 5. Tests
```

✅ WHAT TO CHECK

1. **Skill name matches the folder.** `name: create-card-and-page` must match `.github/skills/create-card-and-page/`
2. **Description is broad enough.** It triggers when anyone mentions adding a new entity — not just “Lore” specifically
3. **No hardcoded entity names.** Every `Lore`, `lore`, `Lores` are replaced with `${entityName}`
4. **Plan steps are still present.** The full implementation roadmap from M2 is preserved, just parameterised

💡 [illegible]
“The description only mentions Lore — broaden it to cover any entity type.”

E5 Step 2 — Improve with Templates & Scripts

ASK COPILOT TO IMPROVE THE SKILL

```
Look at .github/skills/create-card-and-
page/SKILL.md.

Let's improve this skill by adding:

1. A templates/ folder with a starter
  scaffold for each file the skill generates.
  Use the existing Lore entity files as
  reference. Replace entity-specific names
  with ${entityName} placeholders.

2. A scripts/ folder with:
  - check-scaffold.ps1 — verifies all
   expected files were generated
  - run-migration.ps1 — applies the
   EF Core migration for the new entity

Update SKILL.md to reference both folders
in the ## Resources section.
```

ITERATE UNTIL IT'S RIGHT

1. Run the skill for **“`Locations`”** — does the plan reference the same files as the Lore plan? If not, describe what's missing and ask Copilot to fix the template.
2. Ask Copilot to run `check-scaffold.ps1` after a test invocation. Let it read the errors and self-correct.
3. If a template generates wrong output, paste the diff and ask: *“This generated X but I expected Y — update the template.”*
4. Keep looping. The goal isn't a perfect first prompt — it's a skill that gets better with each round of feedback.

💡 **Copilot as collaborator, not oracle.** [illegible]

E5 — Inside create-card-and-page skill

```
---
name: create-card-and-page
description: >
  Use this skill when asked to add a new
  entity type with a card grid page to FanHub.
  It scaffolds the full stack: backend model,
  DbContext, seed data, EF Core migration,
  REST controller, frontend model, stat card,
  nav-card, and a Blazor list page.
---

## What to ask before starting
1. Entity name — singular PascalCase
2. Properties beyond Id and ShowId
3. Form fields for the add form
4. Nav card label (plural display)
5. Nav card icon (emoji)
6. Seed source file
```

📁 Skill Directory

```
.github/skills/create-card-and-page/
  ├── SKILL.md
  ├── templates/
  │   ├── BackendModel.cs
  │   ├── Controller.cs
  │   ├── FrontendModel.cs
  │   ├── SeedBlock.cs
  │   ├── HomeStatCard.razor.txt
  │   ├── HomeNavCard.razor.txt
  │   └── ListPage.razor.txt
  └── scripts/
      ├── check-scaffold.ps1
      ├── run-migration.ps1
      └── verify-endpoints.ps1
```

💡 **Not just instructions.** [illegible]

🧩 Templates — The Factory Floor

TEMPLATES/BACKENDMODEL.CS

```
namespace Backend.Models;

public class { EntityName }
{
  public int Id { get; set; }
  public int ShowId { get; set; }
  // Add requested properties here
  // No navigation props, no data annotations
  // All strings non-nullable
  public string Title { get; set; }
  public string Description { get; set; }
}
```

COPILOT ASKS, THEN GENERATES

- Entity: `Lore`
- Properties: `Name, Description`
- Nav label: `"Lore"`
- Icon: 📍
- Seed: `breaking-bad-universe.md`

↓ generated

```
namespace Backend.Models;

public class Lore
{
  public int Id { get; set; }
  public int ShowId { get; set; }
  public string Name { get; set; }
  public string Description { get; set; }
}
```

💡 **7 templates, one parameter.** Every file — backend, frontend, controller, seed, 3 React components — [illegible] answers. Change `Lore` to `Ratings` and the whole stack regenerates.

🔄 9 Steps, 3 Scripts

⚙️ BACKEND

1. Backend model `{EntityName}.cs`
2. Register DbSet in FanHubContext
3. Seed block from universe file (12–15 records)

🧯 SCRIPTS

`check-scaffold.ps1`

Verifies 6 files exist, DbSet registered, seed block present — before migration runs

`↓ step 3.5 gate`

`run-migration.ps1`

Runs EF Core migration with clear error reporting. Stops the skill if it fails.

`↓ step 4`

🖥️ FRONTEND

5. REST controller (GET + POST)
6. Frontend model mirroring backend
7. Stat card on Home.razor
8. Nav card on Home.razor
9. Blazor list page + inline add form

`verify-endpoints.ps1` — Confirms GET + POST work: [illegible] record. Run after the backend is up.

▶️ Running new-card-skill for Locations

💬 INVOKE THE SKILL

Ask Copilot in any file — it picks up the skill automatically:

```
Add a new Locations entity to FanHub.
Entity: Location
Properties: Name, Description, Address
Nav label: "Locations"
Icon: 📍
Seed: breaking-bad-universe.md
```

Or just say `"add a Locations card to FanHub"` *— the skill triggers and asks for what it needs.*

📋 EXPECTED OUTPUT

Copilot generates a scoped plan covering:

- ✓ `Backend/Models/Location.cs`
- ✓ `Backend/Controllers/LocationController.cs`
- ✓ `Frontend/Models/Location.cs`
- ✓ `Components/LocationStatCard.razor`
- ✓ `Components/LocationNavCard.razor`
- ✓ `Pages/Locations.razor`
- ✓ DbContext entry + EF migration + seed data

Same 9-step plan as Lore and Ratings — just with `Location` *everywhere* `Lore` *was.*

> *"I said 'add a Locations card.' It came back with six files, a migration, and seed data. I said four words."*
>
> — Jessica

🔗 Compounding Value

You've built two layers. They multiply each other.

Prompts capture intent. Skills make [illegible] to Copilot. Together they become a workflow layer that grows with the team.

Prompt files (M3)

[illegible]

- Best thinking preserved as team assets
- Consistent outputs across teammates
- Version-controlled & discoverable

Agent skills (M4)

[illegible]

- Copilot reaches for the right behavior
- Templates guarantee consistent output
- Scripts validate and verify each run

Modules 5 & 6

BUILT ON THIS FOUNDATION

- **Module 5:** stable skills become integration entry points
- **Module 6:** skills + prompts power trustworthy custom agents

Every future feature starts with a prompt. The best ones become skills — and the whole team benefits automatically.

✅ Module Checklist

Six exercises. Two layers. One complete workflow foundation.

📝 PART 1 — PROMPT FILES (E1–E3)

E1 Refresh the source of truth

- `refresh-docs.prompt.md` created
- [illegible] and stays aligned after

E2 Save one strong prompt

- [illegible] promoted
- [illegible] — saved & reusable

E3 Encode domain knowledge

- `check-data-accuracy.prompt.md`
- [illegible] entity type, MCP-ready for [illegible]

⚡ PART 2 — AGENT SKILLS (E4–E5)

E4 Graduate a prompt to a skill

- `check-data-accuracy` [illegible] via `/create-skill`
- [illegible] automatically, no manual invocation needed

E5 Card & page skill

- [illegible] `create-card-and-page` skill
- [illegible] in 30 seconds

Later modules get better because the team knows how to package repeatable work into invokable, trustworthy [illegible]

Keep the Foundation Evergreen

Before Module 4, document your new prompt toolkit.

Update `ARCHITECTURE.md` and `.github/copilot-instructions.md` so the skills you build next know the prompts already exist.

`You already built the tool. Run it.`

`/refresh-docs`

The `refresh-docs.prompt.md` you built in Exercise 3.1 updates `ARCHITECTURE.md` and `copilot-instructions.md` with the new prompt files — so Modules 4, 5, and 6 know they exist.

🔁 BUILD THE HABIT

Run `/refresh-docs` after every meaningful change.

Each PR that updates docs keeps the foundation relevant — so Copilot always has accurate context to work from.

SDP

Module 3+4 Is Locked In

🔌 Next Up: Module 5 — MCP Servers

📝

You Captured Repetition

High-value prompts are now saved as reusable team entry points

⚡

You Packaged Expertise

Skills embed domain knowledge and workflows the whole team can invoke

🔌

Now Connect Live Systems

Module 5 moves from packaged local workflows into runtime integrations

Prompt files make repeated work reusable. **Agent skills go further — Copilot itself can reach for them, without being asked.** MCP servers will let it bring live systems into that same workflow.

💬 Reflect & Discuss

📖 STORY MOMENTS

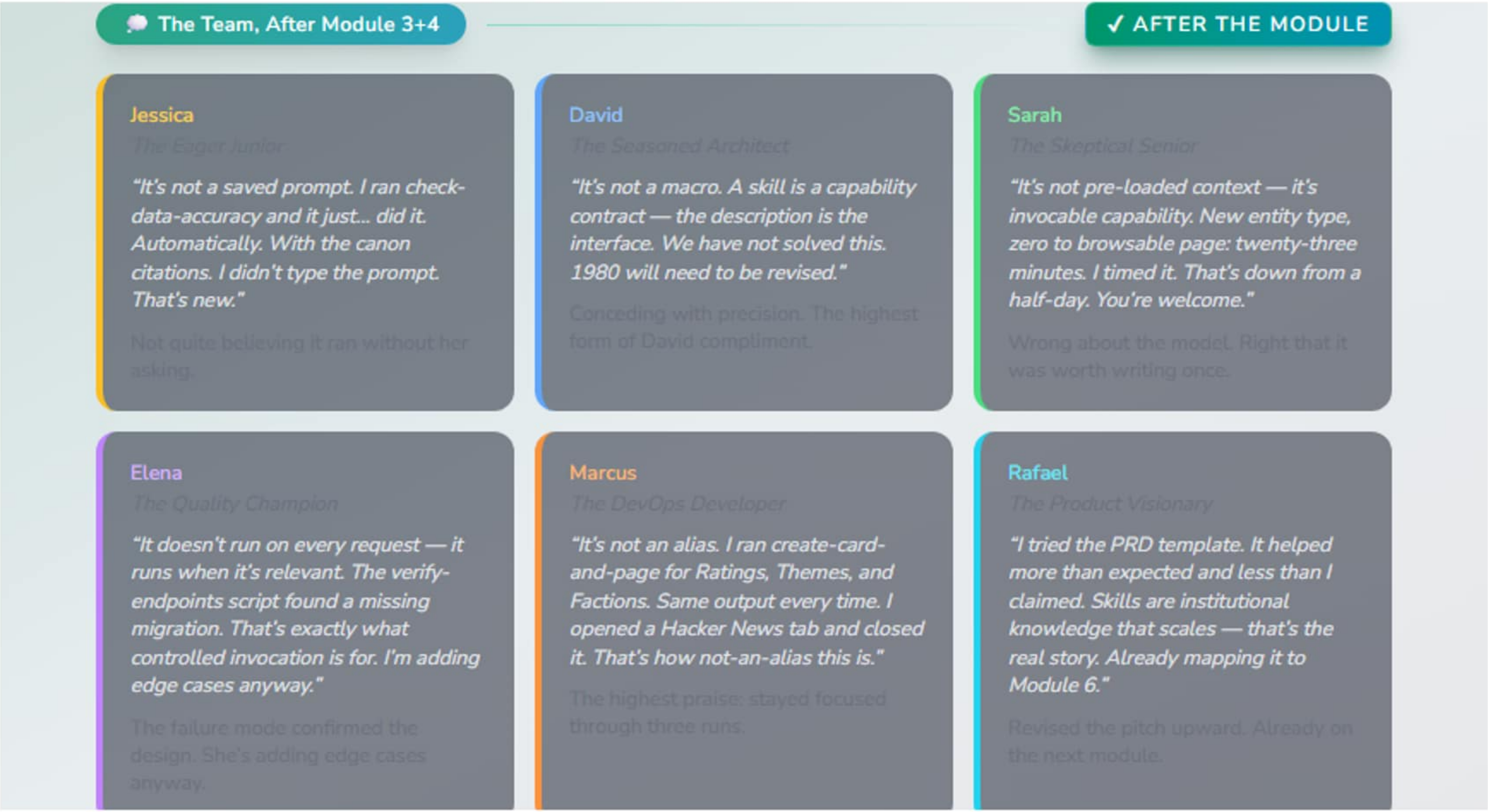
Marcus discovered that `/plan` now runs automatically — the skill removed the decision entirely, not just the typing

David had to *explain* the Walter White error to the skill — it caught a show-accuracy mistake David would have defended if asked

Jessica's realization: "it checked and it was right" — the app had grown a defense mechanism nobody has to maintain

💡 ONE BIG QUESTION

What's one check you do manually on every PR or story — something so routine you barely notice it? Could that become a skill?



💭 The Team, After Module 3+4

✓ AFTER THE MODULE

Jessica

The Eager Junior

“It’s not a saved prompt. I ran check-data-accuracy and it just… did it. Automatically. With the canon citations. I didn’t type the prompt. That’s new.”

Not quite believing it ran without her asking.

David

The Seasoned Architect

“It’s not a macro. A skill is a capability contract — the description is the interface. We have not solved this. 1980 will need to be revised.”

Conceding with precision. The highest form of David compliment.

Sarah

The Skeptical Senior

“It’s not pre-loaded context — it’s invocable capability. New entity type, zero to browsable page: twenty-three minutes. I timed it. That’s down from a half-day. You’re welcome.”

Wrong about the model. Right that it was worth writing once.

Elena

The Quality Champion

“It doesn’t run on every request — it runs when it’s relevant. The verify-endpoints script found a missing migration. That’s exactly what controlled invocation is for. I’m adding edge cases anyway.”

The failure mode confirmed the design. She’s adding edge cases anyway.

Marcus

The DevOps Developer

“It’s not an alias. I ran create-card-and-page for Ratings, Themes, and Factions. Same output every time. I opened a Hacker News tab and closed it. That’s how not-an-alias this is.”

The highest praise: stayed focused through three runs.

Rafael

The Product Visionary

“I tried the PRD template. It helped more than expected and less than I claimed. Skills are institutional knowledge that scales — that’s the real story. Already mapping it to Module 6.”

Revised the pitch upward. Already on the next module.